

[Marcar como hecha](#)

11.7. Opciones de configuración del archivo web.xml

De momento hemos visto el modo de crear automáticamente en la carpeta WEB-INF, el web.xml

Pero este documento es muy relevante en nuestra aplicación. Vamos a verlo

Configuración de servlets

En el API Java Servlet, los objetos `javax.servlet.ServletConfig` representan la configuración de un servlet. La información de configuración contiene parámetros de inicialización (un conjunto de pares nombre/valor), el nombre del servlet y un objeto `javax.servlet.ServletContext`, que ofrece la información del servlet sobre el contenedor. Los parámetros de inicialización y el nombre de un servlet pueden ser especificados en el descriptor de despliegue (el archivo web.xml), por ejemplo:

```
<web-app>
<servlet>
<servlet-name>Admin</servlet-name>
<servlet-class>com.aps.admin.AdminServlet</servlet-class>
<init-param>
<param-name>email</param-name>
<param-value>adm@admin.es</param-value>
</init-param>
<init-param>
<param-name>helpURL</param-name>
<param-value>admin.aps.com</param-value>
</init-param>
</servlet>
</web-app>
```

Este ejemplo registra un servlet de nombre admin y especifica dos parámetros de inicialización, email y helpURL. El contenedor Web lee esta información, y la pone a disposición de `com.aps.admin.AdminServlet` mediante el objeto `javax.servlet.ServletConfig`. Si queremos cambiar estos parámetros, podemos hacerlo sin tener que volver a compilar el servlet.

El mecanismo del descriptor de despliegue fue introducido en la especificación Servlet 2.2. Los contenedores Web (llamados entonces motores de servlet) que se ajustan a versiones antiguas de la especificación, proporcionan medios específicos del proveedor (como archivos apropiados) para especificar los parámetros de inicialización.

La interfaz ServletConfig

Esta interfaz especifica los siguientes métodos:

- `public String getInitParameter (String name)`
- `public Enumeration getInitParameterNames()`
- `public ServletContext getServletContext ()`
- `public String getServletName ()`

El método `getInitParameter()`

```
public String getInitParameter (String name)
```

Este método devuelve el valor de un parámetro llamado inicialización o null si el parámetro especificado no existe. En el ejemplo anterior, al invocar el método `getInitParameter` con "email" como argumento, se obtiene el valor "admin@admin.es".

El método `getInitParameterNames()`

```
public Enumeration getInitParameterNames ()
```

Este método devuelve una enumeración de todos los parámetros de inicialización de un servlet. Utilizando esta enumeración, puede obtener los

?

nombres de todos los parámetros de enumeración sucesivamente. Si no hay parámetros de inicialización especificados, este método devuelve una enumeración vacía. En el ejemplo anterior, invocar el método `getInitParameterNames()` devuelve una enumeración que contiene dos objetos `String`: "email" y "helpURL".

El método `getServletContext()`

```
public ServletContext getServletContext ()
```

Este método devuelve una referencia al objeto `ServletContext` asociado con la aplicación `MiAplicación.ServletContext`

El método `getServletName()`

```
public String getServletName ()
```

Este método devuelve el nombre asignado a un servlet en su descriptor de despliegue. Si no hay ningún nombre especificado, éste devuelve en su lugar el nombre de clase de servlet.

En teoría, no hay nada que impida que un contenedor Web ejecute el ciclo de vida completo del servlet cada vez que un servlet es solicitado. En la práctica, los contenedores Web cargan e inicializan los servlets durante el arranque del contenedor o cuando el servlet es invocado por primera vez y mantienen esa instancia de servlet en la memoria para revisar todas las solicitudes que recibe. El contenedor puede decidir en cualquier momento liberar la referencia de servlet, finalizando así el ciclo de vida del servlet. Esto podría ocurrir, por ejemplo, si el servlet no ha sido invocado durante un tiempo o si el contenedor se está cerrando. Cuando esto sucede, el contenedor invoca el método `destroy()`.

En el modelo habitual de ciclo de vida de servlet, el contenedor Web crea una única instancia de cada servlet. Pero, ¿qué ocurre si el método `service()` del servlet está todavía ejecutándose cuando el contenedor Web recibe otra solicitud? Para los servlets que no implementan la interfaz `javax.servlet.SingleThreadModel`, el contenedor invoca la misma instancia de servlet en cada hilo de solicitud. Por lo tanto, siempre es posible que el método `service()` sea ejecutado en más de un hilo, siendo necesario que el método `service()` esté a salvo de hilos. Aparte de no acceder a recursos a salvo de hilos (como escribir a archivos), debemos también considerar mantener nuestros servlets sin estado (es decir, no definir ningún atributo en sus clases de servlet). Al definir variables de instancias en nuestros servlets, debemos asegurarnos de que tales variables son manipuladas de modo seguro.

Última modificación: lunes, 27 de febrero de 2023, 19:01